

Lesson 9

Advanced State — `useReducer` & `useContext`

Code the Dream — React

Today's Session

EXPLORE

- The problem with many `useState` calls
- Actions, reducers, and dispatch
- `useReducer` — syntax and mental model
- `useState` VS. `useReducer` — when to use each
- Prop drilling and the `useContext` solution
- `useReducer` + `useContext` combined pattern

APPLY

- Extend a Survey Builder MVP built with `useReducer` + `useContext`
- Add 5 new reducer actions for editing and deleting
- Wire new UI components to dispatch those actions

By the end: model complex state as reducers, share state across components without prop drilling, and extend a real codebase that already uses these patterns.

The Problem: Many `useState` Calls

As apps grow, related state can scatter across many hooks:

```
const [cart, setCart] = useState([]);  
const [isCartOpen, setIsCartOpen] = useState(false);  
const [isCartSyncing, setIsCartSyncing] = useState(false);  
const [cartError, setCartError] = useState('');
```

Problems this creates:

- State update logic is spread across multiple handlers
- Related values can fall out of sync (e.g. forgetting to reset error when the cart closes)
- Hard to trace what caused a specific state change

When several state variables always change together or represent one "thing," that's a signal to consolidate them with `useReducer`.

The Three Parts of the Reducer Pattern

Action — a plain object describing what happened

```
{ type: 'ADD_ITEM', payload: { id: 'p1', name: 'T-Shirt', price: 19.99 } }
```

Reducer — a pure function: receives (state, action), returns new state

```
function cartReducer(state, action) {  
  switch (action.type) {  
    case 'ADD_ITEM':  
      return { ...state, cart: [...state.cart, action.payload] };  
    default:  
      return state;  
  }  
}
```

Dispatch — sends an action to the reducer (replaces calling a setter directly)

```
dispatch({ type: 'ADD_ITEM', payload: item }); // instead of setCart([...cart, item])
```

useReducer — Syntax

```
const [state, dispatch] = useReducer(reducer, initialState);
```

state replaces multiple useState values. dispatch replaces individual setters.

```
const initialState = {  
  cart: [],  
  isCartOpen: false,  
  isCartSyncing: false,  
  error: '',  
};
```

```
const [cartState, dispatch] = useReducer(cartReducer, initialState);  
// Access values: cartState.cart, cartState.isCartOpen, etc.  
// Trigger changes: dispatch({ type: 'open' })
```

Place the reducer function and `initialState` in a separate file (e.g. `cart.reducer.js`) — they don't depend on the component and the component file stays small.

Reducer Cases — The Pattern

Each `case` block receives the full state, spreads it into a new object, and overrides only what changed.

```
case 'open':  
  return { ...state, isCartOpen: true };  
  
case 'add':  
  return { ...state, cart: [...state.cart, action.payload] };
```

- Spread `...state` first to keep all unchanged fields
- Override only the properties that should change
- Always return a new object — never mutate the existing state

Include a default: return state at the bottom of every switch — without it, an unknown action type returns undefined and crashes the app.

Reducer Cases — Full Example

```
function cartReducer(state, action) {  
  switch (action.type) {  
    case 'open':  
      return { ...state, isCartOpen: true };  
    case 'close':  
      return { ...state, isCartOpen: false };  
    case 'add':  
      return { ...state, cart: [...state.cart, action.payload] };  
    case 'error':  
      return { ...state, error: action.error };  
    default:  
      return state;  
  }  
}
```

Dispatching `{ type: 'add', payload: item }` hits the 'add' case, which returns a new state object with the item appended — leaving `isCartOpen`, `isCartSyncing`, and `error` unchanged.

Check for Understanding

A component uses `const [cartState, dispatch] = useReducer(cartReducer, initialState)`. The user clicks a "Close Cart" button. What should the click handler do?

- A) `setIsCartOpen(false)` — update the state variable directly
- B) `dispatch({ type: 'close' })` — send an action to the reducer
- C) `cartState.isCartOpen = false` — mutate the state property
- D) `return { ...state, isCartOpen: false }` — call the reducer logic directly

useState VS. useReducer — When to Use Each

	useState	useReducer
Best for	Simple, independent values	Related values that update together
Update logic	Spread across handlers	Centralized in one reducer
Testing	Logic lives inside component	Reducer is a pure function — easy to unit test
Boilerplate	Minimal	More setup
Use when	1–2 state vars, simple updates	3+ related vars, complex or coordinated updates

Start with `useState`. Reach for `useReducer` when you notice yourself calling multiple setters in every handler, or when state updates depend on each other in complex ways.

Prop Drilling — The Problem

Data that lives at the top of the tree must be passed down through every component that doesn't need it, just to reach the one that does.

```
App (owns theme + toggleTheme)
├── Layout      ← receives theme, passes it down
│   └── Sidebar ← receives theme, passes it down
│       └── ThemeToggle ← finally uses theme
```

Every intermediate component gets props it doesn't use. This clutters interfaces and makes components harder to reuse.

If a prop passes through 2+ components without being used, that's prop drilling. `useContext` lets any descendant read it directly — no middlemen.

Step 1 — Create the Context

Create a context object in its own file and export it so any component can import it.

```
// ThemeContext.js
import { createContext } from 'react';

export const ThemeContext = createContext(null);
```

The `null` argument is the default value — used only when a component tries to read context outside of a Provider. In practice, you always wrap with a Provider.

One file per context keeps things organized. Export the context object itself — the Provider and any custom hooks can live in the same file.

Step 2 — Provide Context to the Tree

Wrap the part of the tree that needs access with a `Provider`. The `value` prop is what every consumer receives.

```
// App.jsx
const [theme, setTheme] = useState('light');
const toggleTheme = () =>
  setTheme(prev => prev === 'light' ? 'dark' : 'light');

return (
  <ThemeContext.Provider value={{ theme, toggleTheme }}>
    <Header />
    <Main />
  </ThemeContext.Provider>
);
```

Both `theme` (the current value) and `toggleTheme` (the updater) are shared. Any component inside the `Provider` — at any depth — can read both without receiving them as props.

Consuming Context with useContext

Any component inside the Provider can call `useContext` to read the shared value directly — no props needed.

```
// Header.jsx – nested anywhere inside <ThemeContext.Provider>
import { useContext } from 'react';
import { ThemeContext } from './ThemeContext';

function Header() {
  const { theme, toggleTheme } = useContext(ThemeContext);
  return (
    <button onClick={toggleTheme}>
      Switch to {theme === 'light' ? 'Dark' : 'Light'} Mode
    </button>
  );
}
```

Header reads `theme` and `toggleTheme` directly — it doesn't receive them as props, and Layout and Sidebar don't need to know about them at all.

useReducer + useContext — The Combined Pattern

Put both `state` and `dispatch` into context — any component can read state or trigger actions without prop drilling either one.

```
// SurveyContext.jsx
export const SurveyContext = createContext(null);

export function SurveyProvider({ children }) {
  const [state, dispatch] = useReducer(surveyReducer, initialState);
  return (
    <SurveyContext.Provider value={{ state, dispatch }}>
      {children}
    </SurveyContext.Provider>
  );
}
```

Any child component accesses state and dispatches actions in one line:

```
const { state, dispatch } = useContext(SurveyContext);
```

Context Performance — The Problem

Every time a context value changes, **all consumers re-render** — even if they only use part of the value.

```
// A new object literal every render → all consumers re-render every render  
<ThemeContext.Provider value={{ theme, toggleTheme }}>
```

Fix: memoize the value so the reference only changes when theme actually changes.

```
const value = useMemo(() => ({ theme, toggleTheme }), [theme]);  
<ThemeContext.Provider value={value}>
```

Context — When to Use It

Good fits for context:

- Auth state and user info
- App-wide theme or locale
- Feature flags
- A reducer's `state` + `dispatch` (stable references)

Bad fits for context:

- Frequently updating state — scroll position, animations, typing input
- State shared by only 2–3 nearby components (just use props)
- Large mixed contexts — split unrelated data into separate contexts

Context is not a replacement for all props. If passing a prop through one intermediate component is the simplest solution, do that.

Now: Apply

Let's extend a production-quality Survey Builder with new features.

This Week's Exercise — Extend the Survey Builder

Open `src/exercises/lesson-09/`. The MVP is already built with `useReducer` + `useContext`.

What already works:

- Create questions (text, multiple-choice, yes/no, rating)
- Add options to multiple-choice questions
- Toggle between edit and preview modes
- Edit the survey title

Client request: *"Can you add the ability to edit questions, edit answer options, and delete questions?"*

Exercise — Three Phases

Phase	Task
1	Explore the MVP — interact with the UI, read the code
2	Add 5 new case blocks to <code>surveyReducer.js</code>
3	Add Edit and Delete UI to <code>QuestionItem</code>

Study the existing reducer cases and component patterns **before writing any new code**. Your additions should look like they belong with what's already there.

Phase 1 — Explore the MVP

Before writing any code, understand what's already there.

Interact with the UI:

- Create a text question, a multiple-choice question, a yes/no, and a rating
- Add options to the multiple-choice question
- Toggle to preview mode
- Edit the survey title

Read the code:

- `surveyReducer.js` — study existing case blocks and the `QUESTION_TYPES` constants
- `SurveyContext.jsx` — see how `useReducer` + `useContext` are wired together
- `components/QuestionItem` — this is where you'll add Edit and Delete UI
- `components/NewQuestion` + `QuestionDialog` — study the dialog pattern

Phase 2 — UPDATE_QUESTION_TEXT

Open `surveyReducer.js`. Add this case to update a question's text by id:

```
case 'UPDATE_QUESTION_TEXT':  
  return {  
    ...state,  
    questions: state.questions.map(q =>  
      q.id === action.payload.id  
        ? { ...q, question: action.payload.newText }  
        : q  
    ),  
  };  
};
```

Use `map()` to find the matching question and return a new object for it; return all others unchanged.

Phase 2 (continued) — DELETE_QUESTION

Use `filter()` to remove the matching question. Also clear the editing state if the deleted question was being edited.

```
case 'DELETE_QUESTION':  
  return {  
    ...state,  
    questions: state.questions.filter(q => q.id !== action.payload.id),  
    ui: {  
      ...state.ui,  
      editingQuestionId:  
        state.ui.editingQuestionId === action.payload.id  
          ? null  
          : state.ui.editingQuestionId,  
    },  
  };
```

The `ui.editingQuestionId` check prevents a stale "editing" highlight on a question that no longer exists.

Phase 2 (continued) — ADD and UPDATE Option Actions

ADD_OPTION_TO_QUESTION — find by `payload.questionId`, append `payload.optionText`:

```
case 'ADD_OPTION_TO_QUESTION':  
  return {  
    ...state,  
    questions: state.questions.map(q =>  
      q.id === action.payload.questionId  
        ? { ...q, options: [...q.options, action.payload.optionText] }  
        : q  
    ),  
  };
```

UPDATE_OPTION_TEXT — find by `questionId`, replace the option at `optionIndex` with `newText`. Use an inner `map((opt, i) => i === optionIndex ? newText : opt)` on `q.options`.

Phase 2 (continued) — DELETE Option Action

DELETE_OPTION_FROM_QUESTION — filter out the option at `payload.optionIndex`, but only if at least 2 options would remain:

```
case 'DELETE_OPTION_FROM_QUESTION':
  return {
    ...state,
    questions: state.questions.map(q => {
      if (q.id !== action.payload.questionId) return q;
      if (q.options.length <= 2) return q; // enforce minimum
      return {
        ...q,
        options: q.options.filter((_, i) => i !== action.payload.optionIndex),
      };
    }),
  };
};
```

The `<= 2` guard prevents deleting an option when only 2 remain — a multiple-choice question needs at least 2 choices.

Phase 3 — Edit Button & Inline Form

Update `components/QuestionItem`. Read state and dispatch from `useContext(SurveyContext)`.

Edit/Cancel button — toggles editing state for this question:

```
<button onClick={() => dispatch({
  type: 'SET_EDITING_QUESTION',
  payload: { id: isEditing ? null : question.id }
})}>
  {isEditing ? 'Cancel' : 'Edit'}
</button>
```

Phase 3 – Continued

Where `isEditing = state.ui.editingQuestionId === question.id`.

Inline edit form — shown only when `isEditing` is true:

- Input pre-filled with current question text (local state)
- "Save" → `dispatch({ type: 'UPDATE_QUESTION_TEXT', payload: { id, newText } })`

Phase 3 (continued) — Delete & Option Editing

Delete button — guard with a confirmation before dispatching:

```
<button onClick={() => {  
  if (window.confirm('Delete this question?')) {  
    dispatch({ type: 'DELETE_QUESTION', payload: { id: question.id } });  
  }  
}}>Delete</button>
```

For multiple-choice questions in edit mode, show each option with:

- Input field for editing the option text
- "Save" → dispatches `UPDATE_OPTION_TEXT` with `questionId` and `optionIndex`
- "Delete" → dispatches `DELETE_OPTION_FROM_QUESTION`; **disable when `options.length <= 2`**
- "+ Add Option" button → dispatches `ADD_OPTION_TO_QUESTION`

No prop drilling — all dispatches go through `useContext(SurveyContext)`. This is the payoff of the combined pattern.

Wrap-Up

This week you learned:

- Related state belongs together — `useReducer` centralizes all update logic in one pure function
- Actions describe what happened; reducers decide what the new state should be; `dispatch` connects them
- Reach for `useReducer` when state has 3+ related values or complex coordinated updates
- `useContext` shares state across any depth of the component tree without prop drilling
- Combining `useReducer` + `useContext` gives you centralized, accessible, testable state management

Before next session: Complete the weekly assignment — refactor your todo app's state into a reducer, provide it through context, and remove prop drilling from your component tree. Submit your PR link in CTD Learns. Post questions in **`#[class-name]-discussion`** on Slack.